

```
{
  "creator" : "ari.neko@cvut.cz",
  "timestamp" : "09-07-2026_17:23:50",
  "variant" : 1,
  "subjectName" : "java-programming",
  "categories" : [ "Generics & Types", "Java Collections Framework", "Object-Oriented Principles" ],
  "questions" : [ {
    "question" : "What does the wildcard bound pattern 'List<? extends Number>' accomplish inside a method signature?",
    "answers" : [ "It turns the List into an immutable collection", "It mandates that all elements in the List must carry matching static values", "It opens the List to accept additions of any custom object structure", "It restricts the parameter to a List containing Number or any of its subclasses" ],
    "correctIndex" : 3
  }, {
    "question" : "What does the wildcard pattern 'List<? super Integer>' signify inside type boundaries checking?",
    "answers" : [ "It limits the collection strictly to elements that implement serializable", "It prevents downstream functions from executing list mutations", "It tells the system that the collection properties are bound to an index", "It specifies a List that can hold Integer or any of its master superclass ancestor layers" ],
    "correctIndex" : 3
  }, {
    "question" : "What happens when you use a raw type instead of a parameterized generic type declaration?",
    "answers" : [ "The Java Virtual Machine executes the bytecode lines 50 percent slower", "You lose compile-time type safety checking benefits and trigger compiler warning logs", "Jackson's ObjectMapper becomes unable to parse JSON resource file streams", "The application crashes instantly with a ClassCastException upon boot paths" ],
    "correctIndex" : 1
  }, {
    "question" : "Which interface inside the Java Collections Framework guarantees that no duplicate elements can be stored?",
    "answers" : [ "Map entries tracker", "List", "Set", "Queue" ],
    "correctIndex" : 2
  }, {
    "question" : "What performance penalty can hit a HashMap if multiple distinct keys yield identical hashCode values?",
    "answers" : [ "The internal memory mapping flushes all cached key allocations", "Hash collisions increase, causing lookup speeds to degrade from O(1) toward O(N)", "The database loader class drops its connection stream pipe instantly", "The JVM throws a ConcurrentModificationException and aborts the thread" ],
    "correctIndex" : 1
  }
}
```

```

}, {
"question" : "What is a major performance difference between an ArrayList and a LinkedList in Java?",
"answers" : [ "ArrayList resizes dynamically without duplicating its internal index arrays", "ArrayList provides O(1) random access time; LinkedList provides O(1) element insertion at boundaries", "LinkedList consumes less memory because it drops link node pointer sets", "LinkedList supports horizontal clustering distributions" ],
"correctIndex" : 1
}, {
"question" : "How do you enforce strict Encapsulation inside a Java POJO bean class definition?",
"answers" : [ "Turn all method trackers static", "Route all object initializations through separate database loaders", "Mark the class structure final and avoid using constructors", "Declare fields private and expose access points strictly via public getter and setter methods" ],
"correctIndex" : 3
}, {
"question" : "What restriction applies to a class declared with the final modifier keyword in Java?",
"answers" : [ "The class cannot contain any public instance initialization constructors", "The internal variables are rendered immutable values instantly", "The JVM forces code tasks to run inside kernel mode", "The class cannot be extended or subclassed by any other class structure" ],
"correctIndex" : 3
}, {
"question" : "Which pillar of OOP allows a subclass to provide a specific implementation of a method defined in its superclass?",
"answers" : [ "Encapsulation", "Abstraction execution rules", "Inheritance (Method Overloading)", "Polymorphism (Method Overriding)" ],
"correctIndex" : 3
}, {
"question" : "What separates method overriding from method overloading in Java programming execution?",
"answers" : [ "Overriding modifies class access modifiers; overloading alters return type bounds", "Overloading is exclusive to abstract interfaces", "Overloading forces garbage collection routines", "Overriding replaces a superclass method at runtime; overloading uses distinct parameters at compile-time" ],
"correctIndex" : 3
}]
}

```